



TRIANGULATING FOR TRUTHINESS

Effective Engagement with
LLMs for Software
Development and
Operations

JONATHAN SNYDER

TRIANGULATING FOR TRUTHINESS

Effective Engagement with LLMs for Software Development and Operations

by Jonathan Snyder

Introduction

It is worse than chaos, it's almost working.

—Anonymous LLM

Effective problem-solving with an LLM requires prioritizing correctness, clarity, and architectural integrity over raw speed or efficiency. Approaching an LLM with the sole intention of “getting things done faster” is a flawed strategy: It tends to produce superficial fixes, hidden bugs, and misaligned solutions. Focusing first on effective, well-validated solutions helps ensure that any efficiency gains are real, sustainable, and do not compromise system reliability or user experience.

While many of the principles outlined in this paper fall into the category of “systems engineering 101,” it is useful to restate them explicitly in the context of working with LLMs. As LLM adoption becomes more widespread in software development and operations, these fundamentals are easy to overlook or shortcut. This paper focuses on the guardrails required to ensure they remain enforced rather than quietly eroded in the pursuit of efficiency.

Caveat Artifex (Let the Creator Beware)

The ideas presented in this paper were developed as part of an experiment I conducted around the start of this year (2026) to see what it would be like to work with LLMs to turn a very basic [React.js](#) application (the [Parametric Geometry Explorer](#)) into something more architecturally robust.

They were written “in collaboration” with LLMs. In a very real sense, they can be considered as *vibe coded guidelines for vibe coding*. They have not been tested or deployed in a “real” software development setting.

Summary

Effective AI “collaboration” requires moving from “giving orders” to expressing intent. Modeled after David Marquet’s concept of Intent-Based Leadership, this framing shifts interaction from task-level instruction to objective-level alignment. System competence is improved iteratively by evaluating an original statement of intent (the objective) against:

- Outputs (quantitative correctness)
- Outcomes (qualitative goal satisfaction)

In other words:

- **Operator verifies everything; LLM provides possibilities:** Always separate human judgment and human decisions from AI suggestions.
- **Codify architectural specifications:** Capture rules, invariants, and contracts as the source of truth.
- **Define clear boundaries and responsibilities in code:** Maintain separation between state, logic, and system layers to prevent hidden dependencies.
- **Reject hacks, timeouts, and speculative fixes:** Quick patches hide underlying problems.
- **Stop guessing; demand data:** Validate all hypotheses with observable evidence before acting.
- **Apply the “five whys” in stages, anchored in available data:** Early guesses guide data collection; final analysis targets actual technical invariants.
- **Test and validate rigorously:** Ensure that user intent, architectural rules, and actual code behavior align; do not treat passing superficial checks or producing green test reports as sufficient validation.

- **Use automation as an Andon cord for AI:** Design tests and checks not only to validate correctness, but to halt work and require human review when changes originating from AI fall outside the scope of what automation can safely evaluate.
- **Treat tests as governance mechanisms when scope is at risk:** The moment tests are expected to fail on unexpected capability expansion they cease to be purely correctness checks and become mechanisms for preserving human responsibility and accountability.
- **Beware the dangers of accretion:** Avoid accumulating undocumented assumptions, temporary patches, or unvalidated “optimizations” or “fixes” that degrade system integrity over time.
- **Move beyond the “vending machine” prompt:** Treat engagement as a sort of Socratic dialog to stress-test assumptions and triangulate truth; the best solution is rarely found in the first suggestion.

Core Philosophy: Governance over Guesswork

I recently gained an insight about LLMs from a conversation with a random stranger while attending a small gathering of technology enthusiasts at a local coffee shop. Unlike traditional compilers or debuggers, LLMs are not deterministic systems. They operate on statistical probability rather than hard logic. This makes them, in a sense, “worse than chaos.” They provide the illusion of precision while harboring a fundamental randomness. For work requiring high predictability and architectural integrity, this unreliability becomes a systemic risk. Therefore, guardrails are required by definition.

To work with an LLM effectively and productively, the operator must provide the external deterministic structure that the model lacks internally and intrinsically. While more recent LLM iterations are connected to some amount of deterministic machine learning infrastructure (e.g., code interpreters, tool-use sandboxes, or constrained decoding), the fact that the LLM is the egress point of today’s massively scaled AI infrastructure is what makes these guardrails so important.

- **Always move from the general to the governed:** As you uncover architectural truths (e.g., how a data pipeline handles out-of-order events), formalize them into architectural specifications. Once a spec is finalized, add it to the LLM’s “custom instructions” or system settings.
- **Rationale:** LLMs are excellent at pattern recall but weak at honoring implicit constraints. Externalizing these rules into explicit specifications transforms the LLM from a “guesser” into a “governed agent.”
- **Critical warning:** LLMs are highly prone to “context drift.” Never assume that communicating a constraint once—even in the same session or in “custom instructions”—is sufficient. Restate, relink, and reverify architectural invariants frequently. Stay alert!

Understanding “Cycle Spinning” vs. Methodical Progress

LLMs often suggest changes based on predictive confidence, not confirmed data. This frequently results in wasted effort and hallucinated fixes. (For more on this, see the section on “Preventing Circular Reasoning in LLM-Assisted Coding” below.)

Push-Back Phenomenon

LLMs may resist requests for additional logging or instrumentation, claiming the root cause is “already clear.”

Truth: A root cause is only verified when the effect of an actual change is observed and traced. Always insist on tracing a value through the full system life cycle, including:

1. User intent
2. Logic handlers
3. Worker/service layers
4. Back to the UI

This ensures every hypothesis is grounded in observable data and prevents acting on assumptions or LLM confidence alone.

Operator Responsibility and Accountability

The human operator must act as the responsible and accountable “epistemic anchor” and the LLM as the well-informed contributor, a “reasoning engine” that must be kept in check through rigorous protocols.

Within this RACI model, it is not just critical for the human operator to have responsibility and accountability, it is equally important to consider the LLM as a key stakeholder to be informed throughout the course of a development project or within the context of an already deployed service. In doing so, it can remain aligned as much as possible to human intent and human decisions, structured and expressed in the form of requirements, strategy, goals, policies, architecture, etc.

Having clarified the correct roles and responsibilities, we can move on to effective problem-solving with an LLM. This requires a staged, data-driven, observation-based approach:

- **Collect evidence first:** Never act on LLM-suggested solutions without validating them against observable system behavior.
- **Use LLMs as hypothesis generators:** LLMs can surface possibilities, patterns, and candidate explanations, but all outputs are speculative until proven with quantitative data (e.g., values match precisely, logs show expected sequences) and observed qualitative results (e.g., the UI renders correctly, the user intent is satisfied).

Focus analysis appropriately:

- **Early stage:** Use broad hypotheses to guide what data or instrumentation is needed.
- **Mid stage:** Identify why sufficient data is still missing and correct false assumptions.
- **Late stage:** Pinpoint technical root causes only when sufficient evidence exists.
- **Double loop learning:** Beyond refining solutions, the operator must periodically re-evaluate whether the current assumptions, problem framings, and

hypotheses are correct in the first place. When progress stalls, this reassessment is often more valuable than further optimization. This is called double loop learning. The scientific method, Scrum, and Lean methodologies such as A3 Thinking all have this feature in common.

While LLMs are effective at iterating within an assumed framing, they are acutely and chronically unreliable at recognizing when that framing is wrong. Detecting and correcting faulty assumptions remains the operator's responsibility.

The Five Whys: Technical Focus is Required

The five whys is a method for exploring the chain of causes behind a problem by repeatedly asking “Why?” at each step. The goal is to identify the violated technical invariant (a bug) or system rule (an architectural/design contract) responsible for the failure. The method is flexible (and iterative): Some problems may be resolvable after fewer than five “whys,” others may require more.

Parametric Geometry Explorer Example #1: Root-Cause Analysis via Technical Invariants

Problem: While in Auto Mode, the heads-up display (HUD) is not updating when the slider changes the pinchAmtX value.

1. Why is the HUD not updating?
 - The Web Worker received a formula that does not reference pinchAmtX.
2. Why does the Worker have a stale formula?
 - [ParametricLogic.js](#) (which sends instructions to the worker) is using settings.uFormula received from [ParametricReducer.js](#) instead of regenerating it from the current scope.
3. Why is the reducer formula stale?
 - The reducer stores only the intent values (pinchAmtX, pinchAmtY), and the derived formula is calculated only when the HUD is first initialized.

4. Why wasn't the formula regenerated for Auto Mode?
 - Auto Mode logic (erroneously) relied on the stale cached string and failed to trigger a refresh upon receiving a new intent value and revision identifier (RID).
5. Why is Auto Mode logic still using the cached string?
 - The Authority Contract Rule 1.1 (Reducer Authority) was not correctly mirrored in the Logic Layer, allowing a stale cache to override the current user intent.

Outcome: The five whys investigation identifies a technical invariant violation going beyond symptoms like “the HUD is broken,” informing a precise fix: Regenerate formulas from the latest scope in Auto Mode whenever user intent changes.

The fix: Enforce the Authority Contract by ensuring the ParametricLogic layer treats the intent state as the primary trigger for formula regeneration, overriding any internal cache when the RID advances.

Reflections On Using the Five Whys with LLMs

Always remember, the causes are never verified until the “fixes” can be observed in the system after a change has been deployed. Furthermore, LLMs may suggest causes outside the technical realm—such as organizational or communication issues. Treat these as context, not diagnosis. They may inform human investigation but must never override an LLM’s evidence-driven technical analysis.

About the importance of raising non-technical problems emerging from technical failures: Repeating patterns of problems often signal deeper issues that cannot be solved by technology alone. Resolving such issues is a management responsibility. Consider situations where management does not recognize this as an “organizational invariant violation.”

Further discussion about how to address this kind of problem goes well beyond the scope of this paper. That said, unless there are effective and safe means of raising non-technical problems while investigating technical failures, chronic patterns of technical failure are likely to persist over the long term.

The Five Whys: Stages of Investigation

The five whys method must be applied in the context of the available data and the stage of investigation. Early in the investigation, “whys” are logical guesses that guide what additional data or instrumentation is needed. Later, they will be refined to focus on why adequate data hasn’t yet been obtained or why a particular hypothesis is demonstrably false or remains potentially true. Only once there is sufficient data can the chain of “whys” focus on the actual root technical cause(s). These are not magical incantations deployed to produce a guaranteed fix at the first sign of a problem.

Stage 1: Initial Failure, Minimal Data (Casting Wide Nets)

- Use broadly focused hypotheses to guide investigation.
- The five whys may involve logically structured guesses to suggest where to collect data next.
- Don’t hesitate to generate multiple sets of five whys and then evaluate and prioritize.

Stage 2: Partial Understanding, Insufficient Data (Funneling).

- Refine analysis to understand why adequate data is not yet available.
- Watch for false positives. A hypothesis may appear correct but could be wrong.
- Redirect the analysis as needed if evidence disproves a previous assumption.
- Look for what is NOT happening. If a hypothesis requires Event X to occur, and the logs show silence, the silence is your primary data point.

Stage 3: Adequate Data (Pinpointing)

- Only when sufficient evidence exists should you apply the five whys to the actual technical root cause(s).
- Remember, the chain may end after three whys or extend beyond five, depending on the complexity and clarity of evidence.
- Never trust LLM statements about a “smoking gun” or “guaranteed fix.” Always validate against observable system behavior.

Stage 4: Reflection (Do the Double Loop)

After a change has been made to the system:

- Establish the chain of causal custody: Did you find a definitive, observable link between cause and effect to assert that the state of the system has been improved? Remember, correlation $\neq$ causation.
- Beware the ghost fix: Be vigilant against “fixes” that appear to work on the surface but either leave the underlying invariant violated or introduce new, hidden regressions.
- Harvest the learning: Once a change is deployed, explicitly consider what was learned. Can this new understanding be codified into the architectural specifications? Does it need to be communicated to other stakeholders (human or AI)?
- The reality of “fixes”: From an objective standpoint, “problems” and “solutions” do not exist in any concrete, measurable sense. We apply these terms to facts and states of affairs when we feel they are more or less desirable. Asserting that something is fixed before it has been deployed and validated can waste time and lead to undesirable consequences. This is a form of hallucination, shared by humans and machines alike.

Preventing Circular Reasoning in LLM-Assisted Coding

This issue most often appears when a problem spans multiple layers (frameworks, run-times, tooling, environment constraints) and the LLM lacks direct observability. Without hard signals, it may repeatedly hypothesize fixes that *sound* plausible but never change the underlying state. Recognizing this early prevents wasted effort and false confidence.

- **Watch for loops:** If an LLM proposes variations of the same fix multiple times, treat that as a diagnostic signal—not forward progress.
- **Pause fixes, gather data:** Circular suggestions usually mean insufficient or ambiguous information. Stop modifying code and collect concrete evidence (logs, runtime values, DOM state, network behavior, test output).
- **Freeze layers deliberately:** Explicitly declare which layers are assumed correct or out of scope (e.g., browser internals, JSDOM behavior, framework guarantees) and reason only at the boundaries your code actually controls.
- **Force convergence:** Reduce the problem until only one adjustable seam remains. Resolve that single uncertainty before reintroducing complexity or exploring additional fixes.
- **Use the five whys to break the loop:** When progress stalls, repeatedly ask why the observed behavior is occurring until you reach a cause that is directly measurable or falsifiable. Asking the LLM to perform its own explicit five whys analysis can be especially useful, as it forces the model to slow down, expose assumptions, and surface where evidence is missing rather than jumping straight to another fix.

On Architecture: Maintain Clear Boundaries and Invariants

Ensure that the roles of state, logic, and side effects are well-defined and separate. The system must be structured so that each component has a clear contract: what it owns,

what it can change, and what it observes.

State must be explicit and predictable.

- **Avoid:** `state.processStatus = "loading ..."` (*ambiguous/transitional state*)
- **Prefer:** `state.status = 'PENDING' | 'SUCCESS' | 'ERROR'` (*discrete, innumerable states*)

Logic must operate on state but not be hidden inside it.

- **Avoid:** `state.updateAndCalculate()` (*logic hidden inside a state object*)
- **Prefer:** `newState = Logic.calculate(state)` (*pure function; explicit transformation, operated from the correct location*)

Side effects must be explicit, observable, and auditable.

- **Avoid:** `function getData() { saveToDB(data); return data; }` (*hidden "fire-and-forget" effect*)
- **Prefer:** `function getData() { const data = fetch(); Logger.log('FETCH', data.id); return data; }` (*logged and traceable*)

Why This Matters When Working with LLMs

LLMs cannot reliably infer hidden architectural rules. While the principles of clear boundaries and invariants may be obvious to human operators, an LLM's focus is limited to local context and statistical probability rather than systemic constraints. This makes explicit architecture critical: Clear boundaries and invariants provide a stable framework against which speculative AI-generated changes can be evaluated and rejected.

In addition to establishing an architectural intent, the architecture must be *operationalized*. In practice, adherence to architecture is enforced through testing and validation mechanisms that encode architectural invariants as executable checks (see the section "Testing as an Architectural Practice"). When those checks are designed not only to validate correctness but also to fail on unexpected scope or capability expansion, they function as governance mechanisms—forming an effective Andon cord for AI (see the section "An Andon Cord for AI: Preserving the Human Touch in AI Automation").

In this way, architecture defines what *ought* to be true, tests assert what *is* true, and Andon-style automation ensures that when the system enters territory it cannot safely evaluate (whether in development or operations), control is explicitly returned to a human operator.

Disciplined Debugging: Evidence Before Conclusions

- **Architectural invariant question:** “What architectural invariant was violated?” Fixes must strengthen system contracts and reinforce deterministic rules, not patch symptoms.
- **Dumb ledger enforcement:** Ensure state storage remains passive. Move any behavior into logic layers.
- **One change at a time:** Unlike human developers, LLMs have no natural resistance to scope expansion. Left unconstrained, they tend to implement multiple changes in parallel. This undermines one of the most important debugging tools available to engineers: the ability to reason about cause and effect through small, isolated changes.
- **Validation before fixing:** Ask: “What is the specific test-to-fail?” Use evidence to guide coding, never speculation.

Testing as an Architectural Practice

A superficial test suite is insufficient to confirm correctness. LLMs tend to generate “Happy Path” tests that pass syntactically without truly stressing the system’s edge cases or architectural boundaries. Specifically, AI-generated tests often:

- **Validate against assumptions, not constraints:** They test scenarios derived from the LLM’s own suggestions rather than the system’s actual requirements.

- **Avoid friction:** They reproduce easily satisfied scenarios and ignore race conditions or high-load failures.
- **Mask fragility:** They produce “green” reports that give a false sense of security while leaving core invariants untested (e.g., by adding a ternary test for null when setting a variable rather than allowing things to fail to surface underlying problems).

Effective validation requires:

- **Inverse validation:** Forcing the LLM to demonstrate how a test fails when a specific architectural contract is violated.
- **Instrumentation over assertions:** Using deep logging to validate rather than merely checking a final output. For example, trace the life cycle of data across threads (worker to UI).
- **Intent alignment:** Confirming that the test measures the architectural invariant, not just the successful completion of a single function.

Remember, operators must treat LLM-provided tests as unverified hypotheses. True reliability is achieved only when the operator forces the LLM to *prove* the test’s efficacy through negative testing, deep instrumentation, and verification against actual system behavior.

An Andon Cord for AI: Preserving the Human Touch in AI Automation

As AI-assisted software development increases in velocity, it becomes difficult for human operators to maintain the necessary and sufficient levels of responsibility and accountability for quality and correctness.

A proven countermeasure is DevOps-style automation. This automation must do more than simply validate correctness: It must also detect when a proposed change falls

outside the scope of what the automation can safely evaluate. When this happens, work must halt and a flag must be raised that signals the need for in-depth human review before progress may continue.

Here I am proposing a kind of Andon Cord for AI: A system that preserves the human touch in automated workflows, ensuring that responsibility and accountability remain explicitly human even in high-velocity environments.

Lean Manufacturing Roots

The Andon cord concept comes from Lean manufacturing and is closely related to the concept of *Jidoka* or *autonomation*—automation with a human touch. In Lean manufacturing, Jidoka ensures that machines stop themselves when something goes wrong so humans can inspect and correct the issue. Similarly, an Andon cord for AI extends the principle to software development and operations: Humans remain the final arbiters of correctness, safety, and scope, even when AI accelerates output.

Parametric Geometry Explorer Example #2: Tests as Permission Boundaries

In retrospect, the approach I took when developing tests for my Parametric Geometry Explorer application aligned (unintentionally) with the principle of Jidoka as it is now being discussed in the context of AI automation. The checks and tests I put in place weren't just validating numeric results or rendering correctness; they evolved in ways that helped define the permissible behavioral envelope of the system.

For example, as they evolved, the automated tests increased the likelihood that geometry changes occurred only through intentional updates, that derived formulas were recalculated whenever parameters changed, and that any AI-suggested new paths or dependencies were surfaced and required explicit approval.

In this sense, the tests I created with AI assistance did more than assert correctness. They acted as operational guardrails—blocking integration and forcing human attention when unexpected or unverified changes appeared—not as a guarantee of correctness or

safety, but as a way to slow the system down and reintroduce human judgment where it mattered.

Testing as a Safety Practice

This type of DevOps automation performs a function already covered in the section on testing above:

- **Assert known invariants:** Check correctness, performance, safety, and contract adherence.

In addition, it adds two critical points for maintaining a safe environment for AI development:

1. **Detect boundary violations:** Identify new files, unexpected dependencies, novel exports, or unapproved behavior paths.
2. **Refuse to proceed when confidence drops:** If the system cannot determine whether a change is safe or compliant, it is halted and flags are raised for human review.

A third point is the most important:

3. **Human accountability is preserved by preventing operators from being overwhelmed into implicit approval:** Without this mechanism, velocity produces unacceptable risk. “It passed CI” can create the illusion of safety even when architectural or behavioral invariants are unknowingly violated.

An Andon cord for AI ensures that velocity becomes disciplined so it can be safely asserted that:

- “The tests passed within the envelope we understand.”
- “Our standards of velocity do not exceed the capabilities of our systems.”

Operational Safety Through Observability and Security

To truly make AI engagement operationally safe, a verified system must maintain and continuously evaluate logs of AI-produced code. Additional checks, inspired by systems designed to detect malicious bugs, viruses, or unintended side effects, also need to be implemented. In other words: Do not trust, and verify.

Finally—and most importantly—observability and security mechanisms must enforce explicit policies defining the systems that may never under any circumstances be reachable, mutable, or even visible to AI-driven workflows. Allowing AI access to the code that defines the boundaries of the systems it is able to access, for example, might lead to unfortunate results.

Broader Significance

AI does more than change how code is written. It fundamentally changes what governance must look like. Teams that treat AI as “just faster typing” risk losing control quietly. Teams must implement explicit, automated stop conditions and preserve human agency as necessary conditions for maintaining accountability and scaling reliability in an era of high-velocity software development.

The Dangers of Accretion

Systems naturally accumulate small patches, assumptions, and workarounds over time. Even well-intentioned fixes can introduce hidden dependencies or violate architectural invariants if not fully validated. When working with LLMs, this risk is amplified: AI-generated suggestions often prioritize local syntactic correctness and immediate plausibility rather than long-term system contracts.

Operators must prevent accretion by ensuring every change is anchored in codified architectural specifications, observable evidence, and core performance invariants (e.g., memory reuse, data ordering). Temporary fixes or speculative solutions must be treated as *architectural debt* and explicitly flagged for replacement once the root cause is identified.

Let's Get Dialogical: Triangulating for Truthiness

The Vending Machine Fallacy

Just as the five whys method *iteratively* sharpens the definition of a problem to reveal its root cause(s), effective LLM engagement is fundamentally a conversation, not a single transaction. Expecting a single prompt to yield a perfect, architecturally sound solution from the outset is a “vending machine” fallacy.

Instead, treat the interaction as a Socratic workshop. Engaging in a sustained dialog—or even cross-referencing a problem across multiple LLMs—leads to superior results by:

- **Stress-testing assumptions:** Forcing the LLM to defend its reasoning reveals logical gaps.
- **Refining the “search grid”:** Each turn in the conversation generates useful information, for example, to narrow the focus from broad hypotheses to specific technical invariants.
- **Triangulating for truthiness:** Comparing how different models approach the same constraint helps identify hallucinations and highlights the most robust architectural paths.

The best answer is rarely the first one to emerge; it is the one that survives a rigorous, multi-turn interrogation and debate.

Local vs. Global Optimization

Never forget, a “technically correct” fix (no syntax errors) can still be a “strategic failure” (violating architectural requirements). The context in which LLMs are deployed drastically changes the quality of their production in respect to both accuracy and precision. This is especially the case with LLMs deployed as code writing helpers. Typically, these LLMs will have indexed access to the code and are focused on providing changes that are technically correct (i.e., good chances of running without error). However, their focus on technical precision comes with a trade-off: Their ability to “understand” and “remember”

intent and context (in the form of strategy, architectural requirements, project goals, etc.) is frequently sacrificed.

Maintaining a dialog not just with LLMs but between LLMs can promote a situation where code generated by an LLM is both “doing things right” and “doing the right things.” This also brings us back to the RACI: Human operators must in all situations maintain responsibility and accountability where AI is involved. Furthermore, establishing dialogs that involve multiple LLMs is not only beneficial in terms of keeping the LLMs honest but keeping you honest. This can be especially helpful when an LLM interaction has left you believing you have produced something especially well-built or innovative.

Implementation Note

Establishing guidelines for working with LLMs is not only important for individuals, teams, and organizations. Sharing the guidelines with LLMs can also help them stay aligned to your goals and requirements (e.g., by adding them to a `[code].cursorsrules` or `system_prompt.md` file). *Obviously and most importantly*, when sharing guidelines with an LLM consider the parts that must be redacted before sharing, e.g., to exclude sensitive security policies.

TABLE 1: How to Start a Conversation (Sample Questions)

Situation	Question to Ask the LLM	Expected Evidence Artifact
Ambiguous Errors	List 3 hypotheses. What log marker would prove each hypothesis to be true or false?	Trace logs, snapshots
Race Conditions	How can Event A always finish before Event B using deterministic async?	Promise chains, callback order
New Features	Which layer owns the state? Which layer handles calculation?	State schema, service method outputs
Performance Lag	Are patterns like Flyweight or Memoization violated?	Profiling, allocation logs

Situation	Question to Ask the LLM	Expected Evidence Artifact
Blind spots	What aspects of the system or environment have not yet been considered that could still cause failures?	Threat Model, Risk Matrix
Running around in circles	What do the guidelines for working with LLMs tell us about what we are doing wrong in approaching this problem?	Process Logs, five whys
Reconnecting to Intent	The smoke tests are passing after that last change you suggested, but aren't the changes introducing a violation of our architectural policy to keep responsibilities separated?	Architecture Spec, ADRs

Don't Forget to Step Outside

Sustained interaction with LLMs increases the risk of epistemic isolation. Repeatedly validating ideas against the same probabilistic systems can reinforce incorrect assumptions rather than expose them. Even when changes are instrumented and tests are passing—even when LLM feedback appears critical and well-considered—the system (you and your AIs) may still be converging on a locally coherent but globally incorrect understanding.

Stepping outside the LLM loop is itself a governance mechanism. This needs to occur informally through dialog with colleagues and formally through well-structured policy and process. Doing so helps surface faulty premises, challenge invisible assumptions, and re-anchor the work to human intent.

Cognitive Offloading

While this section is speculative, it is grounded in well-established human factors research. A significant portion of poor engineering decisions are not made because operators lack skill, knowledge, or intent. They are made because operators are exhausted.

Software systems—especially those developed under time pressure and high complexity—require sustained attention, working memory, and disciplined reasoning. As cognitive load increases, humans become more susceptible to shortcuts, premature conclusions, and invariant violations, even when they know better. This is not a moral failure; it is a physiological and psychological constraint.

Modern automation, including AI-assisted development tools, dramatically increases throughput. Without compensating mechanisms, it also increases the cognitive burden on human operators: more changes to review, more diffs to understand, more plausible-but-incorrect explanations to evaluate. Left unchecked, this creates conditions in which depleted human judgment becomes the weakest link in the system.

There is a potential systems-level benefit to offloading certain forms of cognitive work to machines—not to replace human responsibility, but to preserve it.

When automation enforces architectural boundaries, encodes invariants as tests, halts execution when uncertainty is detected, and requires explicit human authorization to proceed, it reduces the amount of continuous vigilance required from the operator. This allows human cognitive capacity to be applied where it is uniquely valuable: creativity, strategic reasoning, risk assessment, design judgment, and ethical responsibility.

In this framing, mechanisms such as testing-as-governance, staged investigation, and Andon cords are not merely technical safeguards. They are cognitive safety systems. They prevent implicit approval under fatigue, overload, or false confidence, and ensure that judgment is exercised with deliberation rather than bypassed in pursuit of the fastest, easiest outcome.

This argument makes no broader claim about the benefits or risks of AI as a technology. It identifies a narrowly defined potential benefit: The use of automation to offload mechanical cognition so that human judgment can be applied intentionally and sustainably. The goal is not faster decisions, but better decisions made under conditions that respect human cognitive limits and, ultimately, human dignity.

Out of Scope

There are obviously many questions relevant to the topic of AI and software development and operations that are not addressed in this paper. A few that immediately come to mind include:

- Software quality issues aside, what are the risks and potential costs (financial, social, ethical, environmental, etc.)?
- What is the best way to help a team get started?
- What additional challenges must be overcome to successfully scale, for example, across teams, orgs, or even companies/institutions? (“Did you hear what their LLM said to our LLM? Scandalous!”)
- What needs to be considered in the context of a previously deployed product or service that is already in a state of ongoing operation?
- Most importantly, what role can AI play when operating under abnormal conditions, for example, in emergency situations?

Conclusion

LLMs are powerful accelerants for work that requires complex thinking. They amplify both good and bad architecture, operational and development practices, and ultimately, customer experiences.

About the Author

For over twenty years, **Jonathan Snyder** has developed and promoted a humanist approach to working with creative and business technologies. As a Senior Manager at Adobe for nearly fifteen years (2005–2019), he was responsible for globally distributed teams working across a wide range of technology services including: Release Management, Problem Management, Service Quality Improvement, as well as Lean/Agile/

DevOps training and transformation. Snyder began his technology career at the Media Center for Art History, Columbia University, developing academic technology in collaboration with the art history faculty. He is currently conducting research on large language models, working to identify the architectural constraints, epistemological frameworks, and empirical telemetry required to safely and efficiently govern autonomous AI execution in real-world environments. Snyder is a graduate of Reed College, where he obtained an interdisciplinary degree in history and philosophy.